

Design Note — Lexical vs. Semantic Retrieval for News Recommendation

CS4.406 Information Retrieval & Extraction — Assignment 1

1 What We Built

News recommendation differs structurally from catalog domains like retail or streaming: item relevance half-life is hours, not months, so item cold-start is the *default* state, not an edge case (Wu et al. 2020; Kruse et al. 2024). This motivates content-based candidate retrieval — lexical (BM25) and semantic (embeddings) — over collaborative filtering.

This project implements both retrieval methods, plus one shared evaluation harness, over MIND and EB-NeRD under a unified schema and temporal-split protocol (ADR-001, ADR-002), so the two methods and two datasets are compared on identical footing rather than via method-specific pipelines. Both methods were pushed through to real Codabench leaderboard submissions on both competitions (§3.5).

2 Choices We Made

Data & splits. Official train/validation splits are used as-is (7-day windows) — the only feasible option given data budgets (ADR-001). A unified three-table schema (articles, impressions, user_history) with mandatory core + dataset-specific optional fields lets one retrieval and evaluation code path run against both datasets (ADR-002).

Lexical retrieval. BM25Okapi over unweighted concatenated title+abstract query text (full click history, stopwords removed), scored as a precomputed sparse weight matrix rather than `rank_bm25`'s naive `get_scores()` — the latter measured at ~10hr projected vs. ~80s actual at MINDsmall-dev scale, a real infeasibility, not a style preference (ADR-005, ADR-006).

Semantic retrieval. One embedding model computed over both datasets (rejecting EB-NeRD's provided embeddings + a separate MIND model, to keep the single-code-path comparison valid) — `paraphrase-multilingual-MiniLM-L12-v2`, chosen over `multilingual-e5-small` after a same-category vs. different-category cosine-similarity discrimination check: MiniLM's gap (0.069, 2.4x ratio) is real and usable; e5-small's `passage:-`prefixed convention compresses nearly all pairs into a narrow 0.75–0.78 band (gap 0.017, ratio 1.02x) — e5's asymmetric query/passage objective doesn't fit this project's symmetric user-profile-vs-article use case (ADR-008). ANN backend is brute-force cosine similarity — 0.99ms/query at MINDsmall scale, 2.44ms/query at MINDlarge's 255,990 users — FAISS stays unjustified at this scale (ADR-008 + Addendum; revisited at 10x scale, §5). User representation is mean-pooled, L2-renormalized embeddings of full click history; zero-history users get an explicit all-zero vector, mirroring BM25's own cold-start reporting posture.

Rejecting EB-NeRD's provided embeddings (above) was argued, not measured, until a follow-up (ADR-008 Addendum, 2026-08-19, same eval code) found the provided artifact wins `recall@K` and every `Q4` accuracy metric by a CI-clear margin (AUC 0.5453 vs. 0.5430) but loses `Diversity@10` — a real trade-off, EB-NeRD-only, not a blanket verdict (§6).

Evaluation. Two distinct questions are measured, not conflated: *recall@K* (does retrieval surface the clicked article out of the *whole* corpus, $K \in \{50, 100, 200\}$) and *Q4's ranking metrics* — AUC, MRR, nDCG@5, nDCG@10, intra-list diversity, novelty, catalog coverage — computed over the candidates *already listed in each impression*. Ties are broken by a deterministic per-impression-seeded pseudo-random permutation (cold users produce exact ties); $K = 10$ for diversity/novelty. Every metric reports 95% bootstrap CIs ($n = 2000$), sliced by warm/cold user (history length $\geq 5 = \text{warm}$), except catalog coverage, reported as a point estimate only — a set-union statistic is mechanically biased under with-replacement resampling (ADR-007).

3 Observations

3.1 Recall@K — candidate generation over the whole corpus

Dataset (split)	Method	recall@50	recall@100	recall@200
MIND-small dev, overall	BM25	0.73%	1.50%	2.62%
MIND-small dev, overall	Embed	0.90%	1.60%	2.78%
MIND-small dev, warm	BM25	0.73%	1.54%	2.73%
MIND-small dev, warm	Embed	0.93%	1.67%	2.92%
MIND-small dev, cold	BM25	0.72%	1.19%	1.78%
MIND-small dev, cold	Embed	0.65%	1.10%	1.81%
EB-NeRD-demo val (warm only*)	BM25	1.01%	2.13%	4.02%
EB-NeRD-demo val (warm only*)	Embed	0.32%	0.94%	2.57%
EB-NeRD-small val (warm only*)	BM25	0.72%	1.44%	2.77%
EB-NeRD-small val	Embed	0.14%	0.43%	1.21%

*EB-NeRD’s active-user-filtered bundles have zero cold-start users (min history length = 5) by construction. Source: `experiments/{bm25,embed}_{mind,ebnerd,ebnerd_small}_2026-08-10/`. Not run at MINDlarge scale — flagged in §6, not approximated.

3.2 Q4 ranking metrics — AUC / MRR / nDCG (bootstrap 95% CI)

Dataset	Method	AUC overall (CI)	AUC warm	AUC cold
MIND-small dev	BM25	0.569 (0.567–0.571)	0.577	0.524
MIND-small dev	Embed	0.634 (0.632–0.636)	0.644	0.574
MINDlarge dev	BM25	0.570 (0.569–0.571)	0.578	0.522
MINDlarge dev	Embed	0.634 (0.633–0.634)	0.643	0.575
EB-NeRD-demo val	BM25	0.533 (0.528–0.537)	— (=overall)	n/a
EB-NeRD-demo val	Embed	0.544 (0.539–0.549)	—	n/a
EB-NeRD-small val	BM25	0.529 (0.527–0.530)	—	n/a
EB-NeRD-small val	Embed	0.543 (0.541–0.544)	—	n/a

On MIND (both scales), embeddings’ AUC CI does not overlap BM25’s on either cohort. The warm/cold gap is *not* similar between methods: BM25’s is $\Delta 0.052$ – 0.056 , embeddings’ is $\Delta 0.068$ – 0.070 — embeddings lift warm AUC by ~ 0.065 – 0.067 over BM25 but cold by only ~ 0.050 – 0.053 , widening the relative warm/cold disparity rather than closing it.

On EB-NeRD, embeddings’ AUC CI also does not overlap BM25’s — a modest (~ 0.011 – 0.014) edge — despite BM25 winning recall@K by 1.6–2.3x on the *same* corpus (§3.1). These measure different things: recall@K asks whether the click is anywhere in the whole catalog; AUC asks how well the method ranks EB-NeRD’s already-curated, much shorter candidate lists (median 9–12 vs. MIND’s 23). A plausible (not isolated) explanation: EB-NeRD’s long median history (81–93 articles) produces a diffuse mean-pooled query that struggles against the whole catalog but still discriminates within a pre-curated short list.

MRR/nDCG@5/nDCG@10: embeddings lead BM25 by a similar margin on MIND (e.g. MINDlarge nDCG@10: 0.390 vs. 0.350) and a smaller, mixed margin on EB-NeRD (e.g. EB-NeRD-small nDCG@10: 0.459 vs. 0.454).

3.3 Diversity / novelty / coverage (K=10, point estimate for coverage)

Dataset	Method	Diversity@10	Novelty@10	Coverage@10
MIND-small dev	BM25	0.837	16.30	0.083
MIND-small dev	Embed	0.827	16.15	0.078
MINDlarge dev	BM25	0.836	18.09	0.067
MINDlarge dev	Embed	0.828	17.76	0.065
EB-NeRD-small val	BM25	0.795	17.17	0.206
EB-NeRD-small val	Embed	0.789	17.19	0.205

BM25 and embeddings sit within ~ 1 point of each other on every beyond-accuracy metric, on every corpus — accuracy differs meaningfully between methods, beyond-accuracy behavior barely does (n up to 376,471 impressions, so a 1-point gap is not automatically noise, but it is not the large, consistent effect the AUC results show).

3.4 System cost

BM25 at MINDlarge scale: 1.70s index build, 23.5MB sparse weight matrix, ~9.8 min projected full retrieval for 255,990 users (ADR-006 Addendum). Embeddings at the same scale: 264.4s to encode+cache all 72,023 articles, ~10.4 min projected full retrieval (ADR-008 Addendum). Both stay local on an 8GB machine at this scale — only EB-NeRD’s *blind test* split (13.5M impressions, no local ground truth) required Kaggle.

3.5 Leaderboard results (Q5)

Competition	Method	Score	Submission ID	Date
MIND (competitions/13967)	Embed (MiniLM)	0.6195	886468	2026-08-12 13:01
MIND (competitions/13967)	Cohort-gated combiner	0.6192	896696	2026-08-22
EB-NeRD (competitions/2469)	Embed (MiniLM)	0.5404	888045	2026-08-13 23:08
EB-NeRD (competitions/2469)	Contrastive vector	0.5404	896072	2026-08-21 13:01

Both used the embedding method — the clear winner on local dev-set AUC for both datasets (§3.2). EB-NeRD’s first score (0.5404) is close to local validation AUC on `ebnerd_small` (0.5430) — a coherence check, not a claim the two are defined identically.

Second MIND submission — cohort-gated combiner, essentially flat. Local validation predicted a real, CI-clear win at every stage checked pre-submission: +0.0027 on MINDsmall-dev (95% CI +0.0018 to +0.0034), +0.0019 on a MINDlarge-dev re-verification (95% CI +0.0015 to +0.0023, ADR-010’s Addendum). The real leaderboard result was **0.6192 vs. 886468’s 0.6195 — a -0.0003 difference**, essentially flat, not the win either local check predicted. No pipeline defect was found (format verified with this project’s standard discipline: exact line count, zero malformed permutations, all 32 N89741-affected impressions individually spot-checked). Most likely explanation: a genuine population difference — MIND’s official dev/test splits are different, non-overlapping calendar weeks (ADR-001) — and this method’s edge was built around properties specific to the MINDlarge-dev population rather than a large, population-independent effect. Same category of finding as the EB-NeRD result below, now observed twice.

Second EB-NeRD submission — contrastive vector, real test set. Both EB-NeRD submissions round to the same Score (0.5404) — checked before trusting it: the two `predictions.txt` differ in checksum, 99.48% of impressions rank differently for the same ID, confirming genuinely independent runs, not a duplicate upload. The per-submission detail view (grouped by date, “50% of the testset”) resolves the tie:

	06-01	06-02	06-03	06-04	06-05	06-06	06-07	06-08	MEAN
MiniLM AUC	0.5374	0.5422	0.5755	0.5444	0.5296	0.5330	0.5316	0.5240	0.5397
Contrastive AUC	0.5597	0.5533	0.5504	0.5226	0.5314	0.5489	0.5231	0.5323	0.5402

Contrastive leads 5/8 days and the mean, but thinly (+0.0005, sign flips three times day-to-day). MRR/nDCG@5/nDCG@10 means agree in direction (0.3553 vs. 0.3452; 0.3898 vs. 0.3825; 0.4695 vs. 0.4615), but honestly this AUC margin is far smaller than the ~**0.0023 CI-clear gap** ADR-008’s Addendum measured locally on `ebnerd_small` (0.5453 vs. 0.5430) — a real but small edge, not a dramatically larger win.

4 Anti-Gaming and Leakage (Q9)

Two separate Q9 obligations, both addressed directly rather than only described.

Serving-time-unavailable features — measured, not just avoided. Novelty’s popularity signal is computed from the **train split only**, Laplace-smoothed, never from the evaluation split (ADR-007). More specifically, EB-NeRD’s raw `articles.parquet` carries three lifetime aggregate fields — `total_inviews`, `total_pageviews`, `total_read_time` — that are unavailable at serving time. `src/datasets/ebnerd.py` never reads them into the unified schema (grep-verified). To go beyond documenting the exclusion, `scripts/run_leakage_ablation.py` measures what including them would have bought: an untuned 50/50 blend of the (already-deployed) embedding score with a min-max-normalized leak signal built from these three fields, evaluated on `ebnerd_small` validation (ADR-009):

Metric	Without leaky features	With leaky features (50/50 blend)	CIs overlap?
AUC	0.5430 (0.5415–0.5445)	0.5662 (0.5646–0.5678)	No
MRR	0.3437 (0.3421–0.3452)	0.3638 (0.3623–0.3653)	No
nDCG@5	0.3804 (0.3784–0.3823)	0.4034 (0.4016–0.4051)	No
nDCG@10	0.4591 (0.4574–0.4607)	0.4783 (0.4767–0.4799)	No

Every metric moves by a CI-clear margin from exposure to these three fields alone (AUC +0.023 absolute, $\sim 4.3\%$ relative), with no tuning of the blend weight. $\sim 48\%$ of articles were median-imputed rather than zero-filled, so this measured gap is plausibly a conservative *underestimate*. Direct, measured evidence — not just a stated rule — for why these fields are absent from the schema entirely (ADR-009).

Leakage-boundary test. `tests/integration/test_leakage.py` asserts no user’s history click occurs at or after their own earliest impression time. **Exists and passes for EB-NeRD** (demo train + validation splits). **Explicitly skip-marked for MIND:** MIND provides only a static per-user article-ID list, no per-click timestamps, so the temporal boundary can’t be independently verified from the data. An acknowledged gap, not an oversight — MIND’s temporal safety instead rests on the dataset’s own documented split construction (ADR-001).

5 Where It Breaks at 10×

Every real scaling failure found in this project so far was invisible at the previously-exercised scale and appeared at roughly a 5–14x jump, not a 2x one — a pattern, not a one-off.

MINDsmall → **MINDlarge** ($\sim 5x$ users: 50,000 → 255,990; $\sim 14x$ exploded impression-candidate rows) hung indefinitely in a per-token Python loop never exercised at MINDsmall’s row counts; the fix’s first version projected to **$\sim 27GB$** on this 8GB machine (measured directly, not estimated) from one Python object per repeated ID string, fixed via categorical dtype (12.5x measured reduction, $\sim 2.2GB$); a second, separate bottleneck (`series.map()` on categorical data, $>1hr$ with zero progress) was found only by profiling the actually-stuck process. **MINDlarge_test (2.37M impressions) → ebnerd_testset (13.5M impressions, 5.7x further)** surfaced a third, structurally identical failure mode: a converter that materialized a full `list[dict]` before writing anything projected to $\sim 16GB$, invisible at the smaller scale. Fixed by making the read path a generator, consumed one row at a time.

Projecting one more 10x from the largest scale currently verified (MINDlarge: 255,990 users / 72,023 articles; EB-NeRD: 13.5M impressions) is inference, not measurement, but the historical pattern above makes two things predictable. (1) BM25/embedding *retrieval time* scales roughly linearly with user count, so 10x users alone projects to ~ 98 min (BM25) and ~ 104 min (embeddings) — still locally feasible on time alone. (2) Brute-force cosine similarity is $O(\text{users} \times \text{articles})$ — if the *corpus* also grows 10x alongside users (plausible at real news-scale, not guaranteed), that is 100x compute, not 10x, projecting ~ 10.4 min to roughly **17 hours** — this would force revisiting ADR-008’s “FAISS unjustified” call. (3) Every real bug found so far shared one shape — a Python-object-per-row or full-materialization pattern breaking past an unexercised scale; nothing here guarantees another unaudited path is safe at a further 10x — a documented risk, not a resolved one.

6 Limitations and Open Questions

- **True (zero-history) cold-start is a shared ceiling**, not something either method solves. MIND’s cold-cohort recall@200 is nearly identical between methods (BM25 1.78% vs. embed 1.81%).
- **EB-NeRD’s cold cohort is structurally empty** in both `demo` and `small` bundles (min history length = 5, by construction) — no warm/cold comparison is possible for EB-NeRD in this project; `ebnerd_large` was never downloaded/verified.
- **Recall@K was never run at MINDlarge scale** (§3.1) — only Q4’s ranking metrics were.
- **The Q9 leakage-boundary test is real but partial** — passes for EB-NeRD, skip-marked for MIND (no per-click timestamps; see §4).
- **The Q9 ablation’s magnitude is specific to an untuned 50/50 blend** (§4) — a real,

CI-clear lower bound, not the maximum a calibrated leaky model could extract.

- **The encoder choice is verified via a category-based discrimination proxy**, not a curated near-duplicate/paraphrase set — a weak but real relevance signal (ADR-008, Confidence: Medium-High).
- **The hand-built Danish+English stopword list** (BM25 query construction) is unvalidated against an NLP resource (ADR-005).
- **The EB-NeRD recall-vs-AUC disagreement isn't fully explained** (§3.2): BM25 wins whole-corpus recall 1.6–2.3x, yet embeddings edge it on the curated candidate-list AUC. The diffuse-mean-pooled-query explanation is plausible, not isolated.
- **EB-NeRD's provided embeddings win accuracy, lose diversity, locally** (ADR-008 Addendum) — §3.5's real-test-set follow-up is thinner (+0.0005 AUC).
- **Local CI-clear validation wins have not reliably transferred to real blind-test results, twice.** MIND's cohort-gated combiner (§3.5) showed CI-clear wins locally (+0.0027 MINDsmall-dev, +0.0019 MINDlarge-dev) then landed essentially flat on the real leaderboard (-0.0003). EB-NeRD's contrastive vector showed the same pattern in miniature (+0.0023 local → +0.0005 real). Neither traced to a pipeline defect; the leading explanation in both cases is that the method's edge was built around properties specific to the validation population that the official, temporally disjoint test split doesn't guarantee will hold. With $N = 2$, a real named pattern about this project's validation methodology, not yet a quantified relationship (ADR-010's 2026-08-22 Addendum).